Ex.No: 01 **Date:**

Install and configure Java Development Kit (JDK), Android Studio, Android SDK, and Xcode. Create and deploy a basic 'Hello World' app on both Android and iOS simulators.

AIM

To install and configure the required development tools (JDK, Android Studio, Android SDK, and Xcode) and to create and run a simple **Hello World** application on both **Android** and **iOS** simulators.

ALGORITHM

Android Application

1. Install **JDK** and set environment variables (JAVA_HOME).
2. Download and install **Android Studio**.
3. During installation, ensure **Android SDK**, Emulator, and required tools are selected.
4. Open Android Studio → Click **New Project**.
5. Choose **Empty Activity** template.
6. Enter:
    - Application Name: *HelloWorld*
    - Language: Java (or Kotlin)
    - Minimum SDK: Select appropriate version.
7. Click **Finish** and wait for project build.
8. Open activity_main.xml and add a TextView with “Hello World”.
9. Start **Android Emulator** (AVD Manager).
10. Click **Run** to deploy the app.
11. Observe output on emulator screen.

iOS Application

1. Install Flutter
2. Download Flutter
3. Setup:
    a. Extract Flutter
    b. Add to PATH: flutter/bin
    c. Verify: flutter doctor
4. Install Xcode (Required for iOS)
5. Install Xcode (Mac only)
6. Configure :
    sudo xcode-select --switch /Applications/Xcode.app
    sudo xcodebuild –runFirstLaunch
7. Accept license: sudo xcodebuild -license accept
8. Run: flutter doctor

PROGRAM

Android – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World"
        android:textSize="24sp" />
</LinearLayout>
```

Android – MainActivity.java

```
package com.example.helloworld;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

iOS - Open project → go to lib/main.dart

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello App',
```

```
      home: Scaffold(
        appBar: AppBar(
          title: Text('Hello App'),        ),

       body: Center(
          child: Text(
            'Hello World!',
            style: TextStyle(fontSize: 24),
          ),
        ),
      ),
    );
  }
}
```

OUTPUT

Android Output

- The Android emulator launches.
- The screen displays:
  "Hello World" at the center.

iOS Output

- The iOS simulator launches.
- The screen displays:
  "Hello, World!" in large text at the center.

RESULT

Thus, the JDK, Android Studio, Android SDK, and Xcode were successfully installed and configured, and a basic **Hello World** application was created and executed successfully on both **Android** and **iOS** simulators using flutter.

Ex.No: 02 **Date:**

Design a user interface using basic widgets: buttons, text fields, image views, and spinners in Android. Create an equivalent UI layout in iOS using Storyboard and UIKit.

AIM

To design a simple user interface using basic UI components such as **Buttons, Text Fields, Image Views, and Spinners** in Android, and to create an equivalent interface in **iOS** using **Storyboard and UIKit**.

ALGORITHM

Android UI Design

1. Open **Android Studio** → Create **New Project**.
2. Select **Empty Activity**.
3. Open activity_main.xml.
4. Add the following widgets inside a LinearLayout/ConstraintLayout:
    - TextView (Title)
    - EditText (for name input)
    - Spinner (dropdown selection)
    - ImageView (display image)
    - Button (submit)
5. Add entries to Spinner in strings.xml.
6. Link widgets to activity using findViewById() (if needed).
7. Run the application in emulator.

iOS UI Design (Storyboard + UIKit)

1. Open **Xcode** → Create new **iOS App**.
2. Open **Main.storyboard**.
3. Drag and drop UI elements from Object Library:
    - Label (Title)
    - Text Field
    - Image View
    - Picker View (acts like Spinner)
    - Button
4. Use **Auto Layout constraints** to position components.
5. Create **IBOutlet** connections to ViewController.swift.
6. Run app in iOS Simulator.

PROGRAM

MainActivity.java

```
package com.example.layout;
import android.os.Bundle;
import android.app.Activity;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

public class MainActivity extends Activity {
  EditText txtData1, txtData2;
  float num1, num2, result1, result2;

  protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Button add = (Button)findViewById(R.id.button1);
add.setOnClickListener(new OnClickListener(){
     public void onClick(View v){
      try {
        txtData1 = (EditText)findViewById(R.id.editText1);
        txtData2 = (EditText)findViewById(R.id.editText2);
        num1 = Float.parseFloat(txtData1.getText().toString());

       num2 = Float.parseFloat(txtData2.getText().toString());
        result1 = num1+num2;
        Toast.makeText(getBaseContext(),"ANSWER:"+result1,
          Toast.LENGTH_SHORT).show();
      } catch(Exception e) {
        Toast.makeText(getBaseContext(),e.getMessage(),
          Toast.LENGTH_SHORT).show();
      }
     }
    });
    Button sub = (Button)findViewById(R.id.button3);
sub.setOnClickListener(new OnClickListener(){
     public void onClick(View v) {
      try {
        txtData1 = (EditText)findViewById(R.id.editText1);
        txtData2 = (EditText)findViewById(R.id.editText2);
        num1 = Float.parseFloat(txtData1.getText().toString());
        num2 = Float.parseFloat(txtData2.getText().toString());
        result2 = num1-num2;
```

```
Toast.makeText(getBaseContext(),"ANSWER:"+result2,
          Toast.LENGTH_SHORT).show();
        } catch(Exception e) {
          Toast.makeText(getBaseContext(),e.getMessage(),
            Toast.LENGTH_SHORT).show();
        }
       }
     });
     Button clear = (Button)findViewById(R.id.button2);
clear.setOnClickListener(new OnClickListener() {
      public void onClick(View v) {
        try {
          txtData1.setText("");
          txtData2.setText("");
        } catch(Exception e) {
          Toast.makeText(getBaseContext(),e.getMessage(),
            Toast.LENGTH_SHORT).show();
        }
       }
     });
  }
}
```

activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:id="@+id/relativeLayout1"
android:layout_width="fill_parent"
android:layout_height="fill_parent">

<LinearLayout android:id="@+id/linearLayout1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentLeft="true"
android:layout_alignParentRight="true"
android:layout_alignParentTop="true">
<TextView android:layout_width="wrap_content"
android:layout_height="wrap_content"
     android:text="ADDITION"
android:layout_gravity="center"
android:textSize="20dp"/>
</LinearLayout>

<LinearLayout android:id="@+id/linearLayout2"
android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"
android:layout_alignParentLeft="true"
android:layout_alignParentRight="true"

android:layout_below="@+id/linearLayout1">
<TextView android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="Enter No 1"/>
<EditText android:id="@+id/editText1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_weight="0.20"
android:inputType="number"/>
</LinearLayout>

<LinearLayout android:id="@+id/linearLayout3"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentLeft="true"
android:layout_alignParentRight="true"
android:layout_below="@+id/linearLayout2">
<TextView android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="Enter No 2"/>
<EditText android:id="@+id/editText2"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_weight="0.20"
android:inputType="number"/>
</LinearLayout>

<LinearLayout android:id="@+id/linearLayout4"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentLeft="true"
android:layout_alignParentRight="true"
android:layout_below="@+id/linearLayout3">
<Button android:id="@+id/button1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_weight="0.50"
android:text="Addition"/>
<Button android:id="@+id/button3"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_weight="0.50"
android:text="Subtraction"/>
<Button android:id="@+id/button2"
android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:layout_weight="0.50"
         android:text="Clear"/>
        </LinearLayout>


        </RelativeLayout>
```

Open lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: LoginPage(),
    );
  }
}

class LoginPage extends StatefulWidget {
  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {

  TextEditingController username = TextEditingController();
  TextEditingController password = TextEditingController();

  String result = "Result";

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Login Page")),
      body: Padding(
        padding: EdgeInsets.all(20),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [

            Text("Login Page", style: TextStyle(fontSize: 24)),

            TextField(
              controller: username,
              decoration: InputDecoration(hintText: "Username"),
            ),

            TextField(
              controller: password,
              obscureText: true,
```

```
          decoration: InputDecoration(hintText: "Password"),
          ),

          SizedBox(height: 20),

          ElevatedButton(
            onPressed: () {
              setState(() {
                if (username.text == "admin" && password.text == "1234") {
                  result = "Login Successful";

        } else {
                  result = "Login Failed";
                }
              });
            },
            child: Text("Login"),
          ),

          SizedBox(height: 20),

          Text(result, style: TextStyle(fontSize: 20)),
        ],
      ),
    ),
  );
 }
}
```

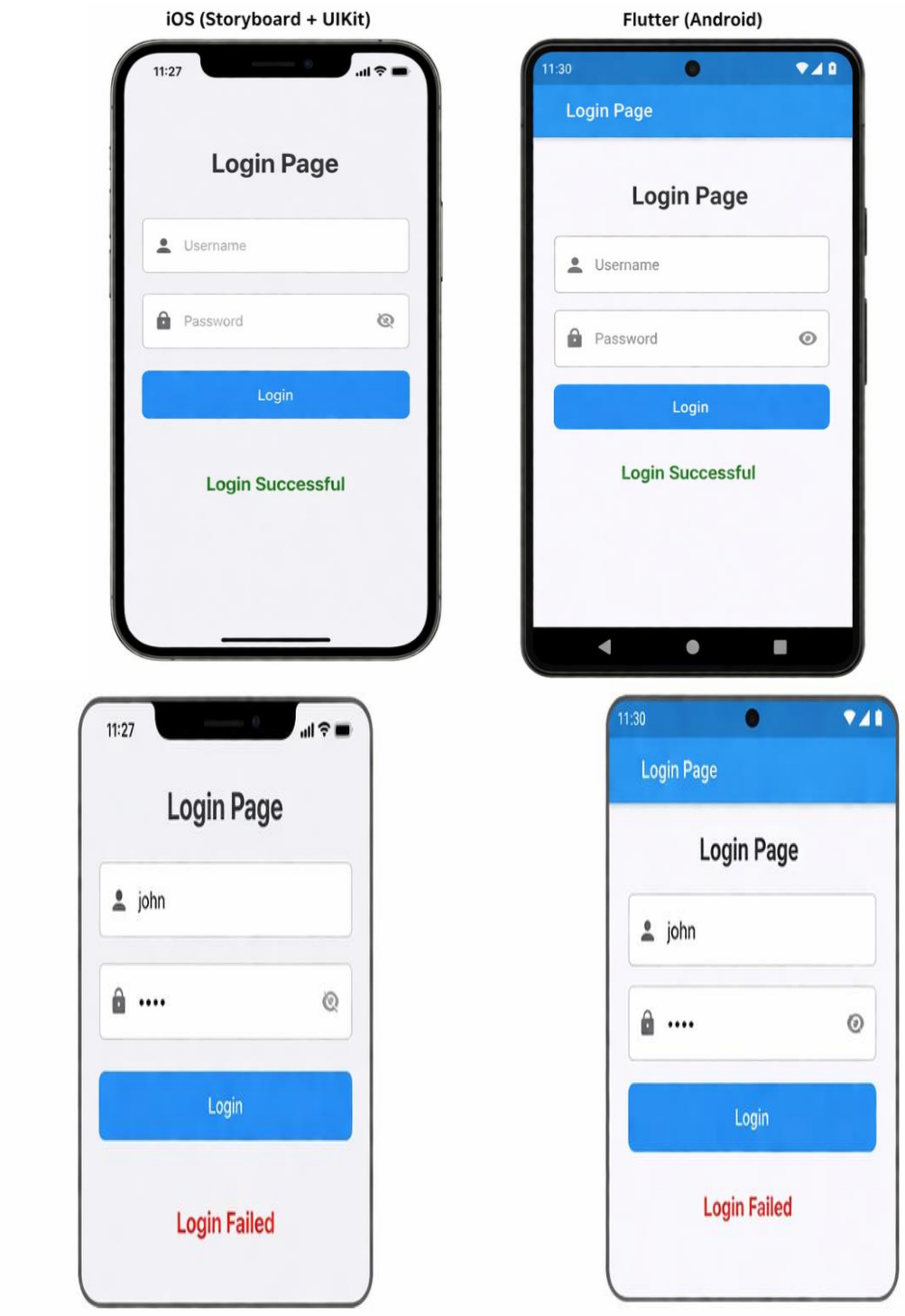
OUTPUT

Android Output

The app screen displays:

iOS Output

The iOS app displays the same interface:



RESULT

Thus, a user interface using basic widgets was successfully designed in **Android**, and an equivalent UI layout was created in **iOS** using **Storyboard and UIKit**.

Ex.No: 03 **Date:**

Develop a mobile app that responds to user interactions using event listeners. Demonstrate layout switching, form submission, and feedback using Android's XML layouts and iOS View Controllers.

AIM

To develop a mobile application that handles **user interaction events**, performs **layout switching/navigation**, processes **form submission**, and displays **feedback messages** in both **Android** and **iOS** platforms.

ALGORITHM

Android Application

1. Create a new project in **Android Studio**.
2. Design **Activity 1 layout** with:
    - EditText (Name input)
    - Button (Submit)
3. Create **Activity 2 layout** to show welcome message.
4. In MainActivity.java:
    - Use **Button Click Listener**.
    - Get user input from EditText.
    - Validate input.
    - Pass data to second activity using **Intent**.
5. In SecondActivity.java:
    - Retrieve data from Intent.
    - Display welcome message.
    - Show **Toast feedback**.
6. Run the app and test interaction.

iOS Application

1. Create new iOS project in **Xcode**.
2. In **Storyboard**:
    - ViewController 1 → TextField + Button
    - ViewController 2 → Label to display message
3. Create segue between controllers.
4. Use **IBAction** for button click.
5. Pass text data to second View Controller.
6. Display alert as feedback.
7. Run app in simulator.

PROGRAM

Android – activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/editName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name" />

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Submit" />
</LinearLayout>
```

Android – MainActivity.java

```
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText editName;
    Button btnSubmit;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        editName = findViewById(R.id.editName);
        btnSubmit = findViewById(R.id.btnSubmit);
      btnSubmit.setOnClickListener(new View.OnClickListener() {
```

```
   @Override
          public void onClick(View v) {
             String name = editName.getText().toString();

             if(name.isEmpty()) {
                Toast.makeText(MainActivity.this, "Please enter your name",
Toast.LENGTH_SHORT).show();
             } else {
                Intent intent = new Intent(MainActivity.this, SecondActivity.class);
                intent.putExtra("username", name);
                startActivity(intent);
             }
          }
       });
   }
}
```

Android – SecondActivity.java

```
import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {

   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_second);

      String name = getIntent().getStringExtra("username");

      TextView tv = findViewById(R.id.textWelcome);
      tv.setText("Welcome " + name + "!");

      Toast.makeText(this, "Form Submitted Successfully", Toast.LENGTH_LONG).show();
   }
}
```

Android – activity_second.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
```

```
   android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <TextView
       android:id="@+id/textWelcome"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:textSize="24sp"/>
</LinearLayout>
```

iOS – ViewController.swift

```
import UIKit

class ViewController: UIViewController {

    @IBOutlet weak var nameTextField: UITextField!

    @IBAction func submitButtonTapped(_ sender: UIButton) {
        if let name = nameTextField.text, !name.isEmpty {
            performSegue(withIdentifier: "showWelcome", sender: name)
        } else {
            let alert = UIAlertController(title: "Error", message: "Please enter your name", preferredStyle:
.alert)
            alert.addAction(UIAlertAction(title: "OK", style: .default))
            present(alert, animated: true)
        }
    }

    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        if segue.identifier == "showWelcome",
           let destinationVC = segue.destination as? SecondViewController,
           let name = sender as? String {
            destinationVC.userName = name
        }
    }
}
```

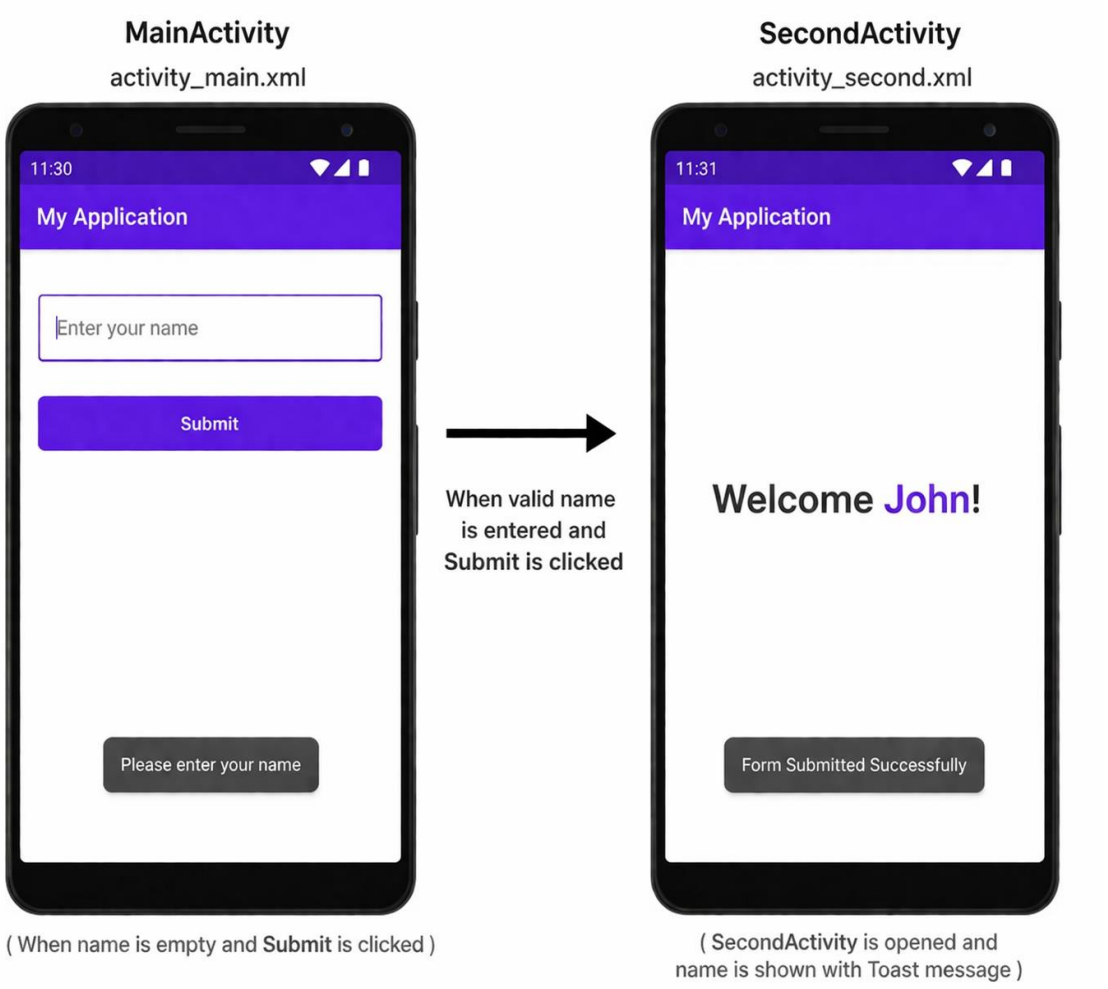
iOS – SecondViewController.swift

```
import UIKit

class SecondViewController: UIViewController {

    var userName: String?
```

```
    @IBOutlet weak var welcomeLabel: UILabel!
   override func viewDidLoad() {
       super.viewDidLoad()
       welcomeLabel.text = "Welcome \(userName ?? "")!"
    }
}
```

OUTPUT

Android



(When name is empty and **Submit** is clicked)

(**SecondActivity** is opened and name is shown with Toast message)

RESULT

Thus, a mobile application was successfully developed that handles **user events**, performs **layout switching**, processes **form submission**, and displays **feedback** in both **Android** and **iOS** platforms.

Ex.No: 04 **Date:**

Implement a four-function calculator that performs addition, subtraction, multiplication, and division. Use platform-specific UI controls, and validate user input in both Android and iOS versions.

AIM

To develop a mobile calculator application that performs **basic arithmetic operations** (Addition, Subtraction, Multiplication, Division) using platform-specific UI controls and implements proper **user input validation** in both Android and iOS.

ALGORITHM

Android Application

1. Open Android Studio → Create New Project.
2. Design layout with:
   - Two EditText fields (Number inputs)
   - Four Buttons (+, −, ×, ÷)
   - TextView to display result
3. In MainActivity.java, assign click listeners to buttons.
4. On button click:
   - Read numbers from EditText
   - Check if fields are empty
   - Convert input to numeric type
   - Perform selected arithmetic operation
   - Check division by zero
   - Display result in TextView
5. Run application.

iOS Application

1. Download **Flutter**
2. Extract and set PATH
3. Run: flutter doctor
4. Create Project
   flutter create calculator_app
   cd calculator_app
   flutter run
5. Understand UI Design
   a. The calculator UI contains
      i. TextFields → input numbers
      ii. 4 Buttons → + − × ÷
      iii. Result Text → display output
      iv. Alert Dialog → validation (iOS style)
6. Write Flutter Code
7. Run the App

PROGRAM

Android – activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <EditText
      android:id="@+id/num1"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter first number"
      android:inputType="numberDecimal" />

   <EditText
      android:id="@+id/num2"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter second number"
      android:inputType="numberDecimal" />

   <Button android:id="@+id/btnAdd" android:text="+"
      android:layout_width="match_parent" android:layout_height="wrap_content"/>

   <Button android:id="@+id/btnSub" android:text="-"
      android:layout_width="match_parent" android:layout_height="wrap_content"/>

   <Button android:id="@+id/btnMul" android:text="*"
      android:layout_width="match_parent" android:layout_height="wrap_content"/>

   <Button android:id="@+id/btnDiv" android:text="/"
      android:layout_width="match_parent" android:layout_height="wrap_content"/>

   <TextView
      android:id="@+id/result"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:textSize="22sp"
      android:text="Result: " />
</LinearLayout>
```

Android – MainActivity.java

```
import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText num1, num2;
    TextView result;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        num1 = findViewById(R.id.num1);
        num2 = findViewById(R.id.num2);
        result = findViewById(R.id.result);

        findViewById(R.id.btnAdd).setOnClickListener(v -> calculate('+'));
        findViewById(R.id.btnSub).setOnClickListener(v -> calculate('-'));
        findViewById(R.id.btnMul).setOnClickListener(v -> calculate('*'));
        findViewById(R.id.btnDiv).setOnClickListener(v -> calculate('/'));
    }

    private void calculate(char operator) {
        String s1 = num1.getText().toString();
        String s2 = num2.getText().toString();

        if (s1.isEmpty() || s2.isEmpty()) {
            Toast.makeText(this, "Enter both numbers", Toast.LENGTH_SHORT).show();
            return;
        }

        double a = Double.parseDouble(s1);
        double b = Double.parseDouble(s2);
        double res = 0;

        switch (operator) {
            case '+': res = a + b; break;
            case '-': res = a - b; break;
            case '*': res = a * b; break;
            case '/':
```

```
 if (b == 0) {

          Toast.makeText(this, "Cannot divide by zero", Toast.LENGTH_SHORT).show();
               return;
             }

     res = a / b;
              break;
       }

       result.setText("Result: " + res);
   }
}
```

Open lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CalculatorPage(),
    );
  }
}

class CalculatorPage extends StatefulWidget {
  @override
  _CalculatorPageState createState() => _CalculatorPageState();
}

class _CalculatorPageState extends State<CalculatorPage> {

  TextEditingController num1 = TextEditingController();
  TextEditingController num2 = TextEditingController();

  String result = "Result";

  // □ Validation Function
  bool validateInputs() {
    if (num1.text.isEmpty || num2.text.isEmpty) {
      showMessage("Please enter both numbers");
      return false;
```

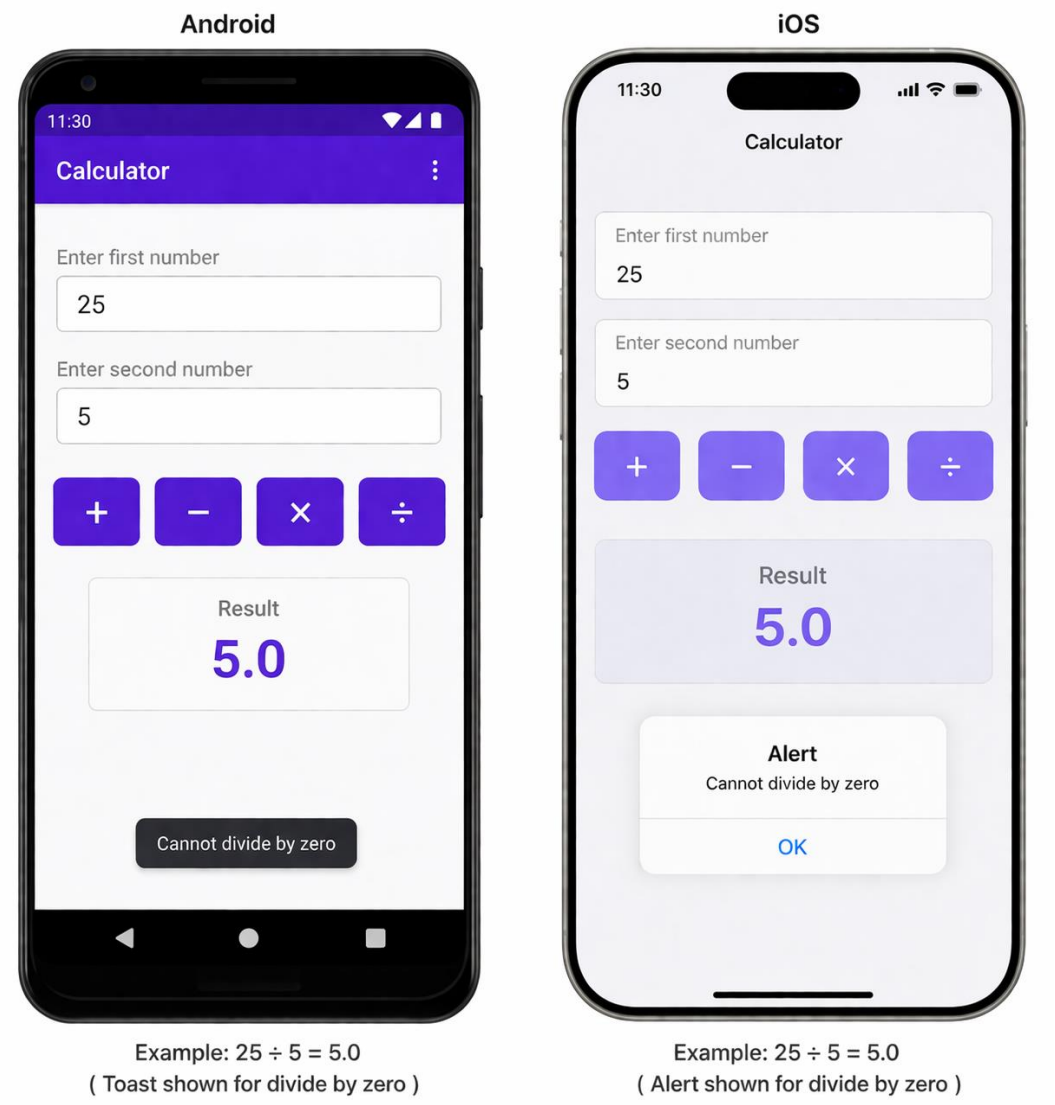
```
  }
    return true;
  }

// □ Show Alert (iOS style compatible)
  void showMessage(String message) {
    showDialog(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: Text("Alert"),
          content: Text(message),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: Text("OK"),
            )
          ],
        );
      },
    );
  }

  // □ Calculator Functions
  void add() {
    if (!validateInputs()) return;
    setState(() {
      result = (double.parse(num1.text) + double.parse(num2.text)).toString();
    });
  }

  void subtract() {
    if (!validateInputs()) return;
    setState(() {
      result = (double.parse(num1.text) - double.parse(num2.text)).toString();
    });
  }

  void multiply() {
    if (!validateInputs()) return;
    setState(() {
      result = (double.parse(num1.text) * double.parse(num2.text)).toString();
    });
  }

  void divide() {
```

```
 if (!validateInputs()) return;

  double second = double.parse(num2.text);
  if (second == 0) {

showMessage("Cannot divide by zero");
    return;
  }

  setState(() {
    result = (double.parse(num1.text) / second).toString();
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("Calculator")),
    body: Padding(
      padding: EdgeInsets.all(20),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [

          TextField(
            controller: num1,
            keyboardType: TextInputType.number,
            decoration: InputDecoration(hintText: "Enter first number"),
          ),

          TextField(
            controller: num2,
            keyboardType: TextInputType.number,
            decoration: InputDecoration(hintText: "Enter second number"),
          ),

          SizedBox(height: 20),

          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton(onPressed: add, child: Text("+")),
              ElevatedButton(onPressed: subtract, child: Text("-")),
              ElevatedButton(onPressed: multiply, child: Text("*")),
              ElevatedButton(onPressed: divide, child: Text("/")),
            ],
          ),
```

```
          SizedBox(height: 20),
          Text(result, style: TextStyle(fontSize: 24)),
        ],
          ),
    ),    );
 } }
```

OUTPUT

Android & iOS



Example: 25 ÷ 5 = 5.0
(Toast shown for divide by zero)

Example: 25 ÷ 5 = 5.0
(Alert shown for divide by zero)

RESULT

Thus, a four-function calculator application was successfully developed with **platform-specific UI controls** and **input validation** in both Android and iOS environments.

Ex.No: 05 **Date:**

Create custom views and animations using Android's Canvas and Animator classes. Implement transitions and interactive animations in iOS using Swift and UIView animation APIs.

AIM

To design **custom UI components** and implement **animations** in Android using **Canvas** and **Animator**, and to create **view transitions and interactive animations** in iOS using **Swift** and **UIView animation APIs**.

ALGORITHM

Android (Custom View + Animation)

1. Create a new Android project.
2. Create a custom class extending View.
3. Override onDraw(Canvas canvas) to draw shapes.
4. Use Paint object to style shapes.
5. Use ValueAnimator or ObjectAnimator for animation.
6. Update position/size during animation.
7. Call invalidate() to redraw view.
8. Run app and observe animated custom object.

iOS (UIView Animations)

1. Create a new project in flutter.
2. Add a UIView (box) in Storyboard.
3. Create IBOutlet for the view.
4. Use UIView.animate() for animation.
5. Change properties like position, size, color, or alpha.
6. Use button action to trigger animation.
7. Run app and observe smooth transition.

PROGRAM

Android – CustomView.java

```
import android.animation.ValueAnimator;
import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Paint;
import android.view.View;

public class CustomView extends View {
```

```
    private Paint paint;
    private float radius = 50;
    public CustomView(Context context) {
        super(context);
        paint = new Paint();
        paint.setColor(0xFF6200EE);
        startAnimation();
    }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawCircle(getWidth()/2, getHeight()/2, radius, paint);
    }

    private void startAnimation() {
        ValueAnimator animator = ValueAnimator.ofFloat(50, 200);
        animator.setDuration(2000);
        animator.setRepeatMode(ValueAnimator.REVERSE);
        animator.setRepeatCount(ValueAnimator.INFINITE);
        animator.addUpdateListener(animation -> {
            radius = (float) animation.getAnimatedValue();
            invalidate(); // redraw
        });
        animator.start();
    }
}
```

Android – MainActivity.java

```
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(new CustomView(this));
    }
}
```

Create Flutter Project

```
flutter create animation_app
cd animation_app
flutter run
```

Implement Animation

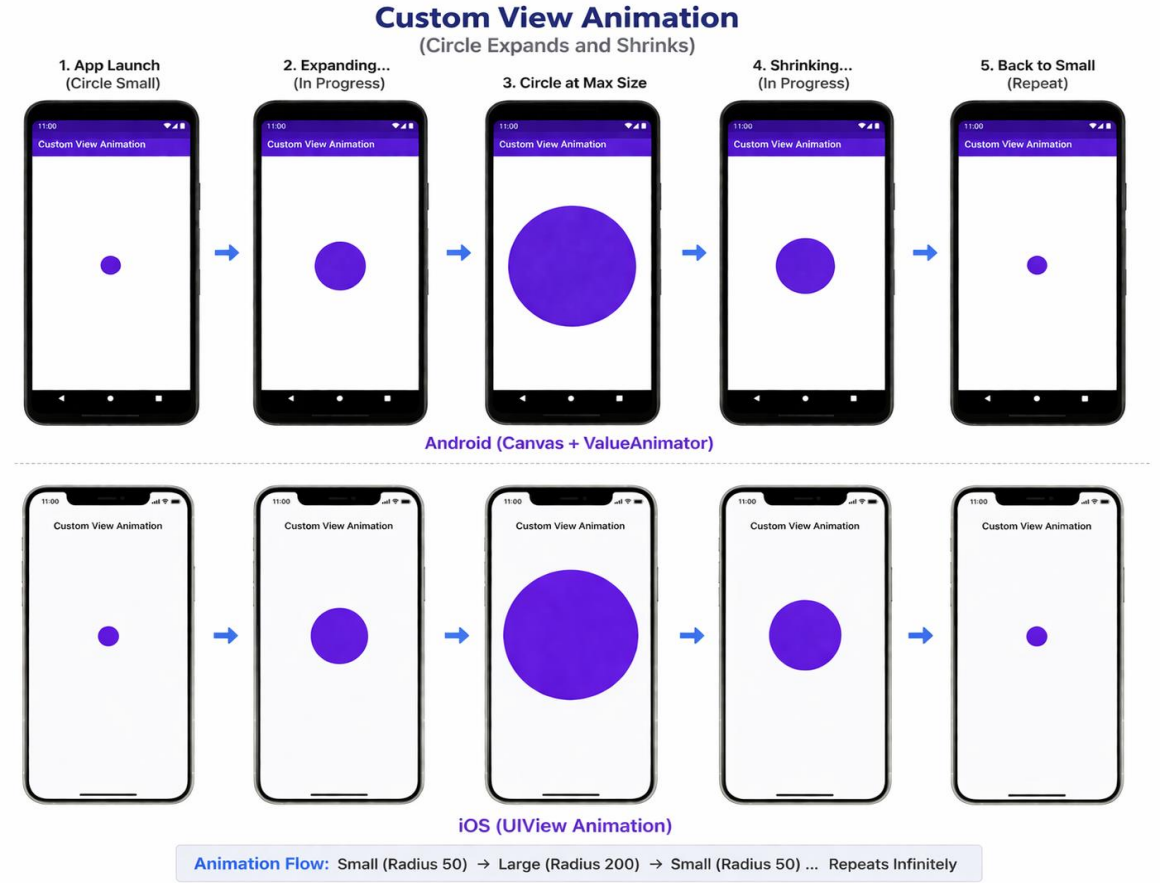
```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: AnimationPage());
  }  }

class AnimationPage extends StatefulWidget {
  @override
  _AnimationPageState createState() => _AnimationPageState();
}

class _AnimationPageState extends State<AnimationPage>
    with SingleTickerProviderStateMixin {

  late AnimationController controller;
  late Animation<double> animation;

  @override
  void initState() {
    super.initState();

    controller = AnimationController(
      vsync: this,
      duration: Duration(seconds: 2),
    )..repeat(reverse: true);

    animation = Tween(begin: 50.0, end: 200.0).animate(controller);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Flutter Animation")),
      body: AnimatedBuilder(
        animation: animation,
        builder: (context, child) {
          return CustomPaint(
            painter: CirclePainter(animation.value),
            child: Container(),
          );        },      ),    );  } }
class CirclePainter extends CustomPainter {
  final double radius;

  CirclePainter(this.radius);
  @override
  void paint(Canvas canvas, Size size) {
```

```
  final paint = Paint()..color = Colors.purple;
    canvas.drawCircle(
    Offset(size.width / 2, size.height / 2),
    radius,
    paint,
  );  }

 @override
 bool shouldRepaint(covariant CustomPainter oldDelegate) => true;  }
```

OUTPUT

Android and iOS Output

- A circle is drawn at the center of the screen.
- The circle continuously **grows and shrinks** (animated radius effect).



RESULT

Thus, custom views and animations were successfully implemented in **Android** using Canvas and Animator classes, and interactive view animations were implemented in **iOS** using Swift and UIView animation APIs.

Ex.No: 06 **Date:**

Create an application to perform insert, update, delete, and display operations using SQLite in Android. Use Core Data or SQLite in iOS for basic CRUD operations with a local database

AIM

To create a mobile application that performs **CRUD operations** on a local database using **SQLite in Android** and **Core Data** in iOS.

ALGORITHM

Android (SQLite)

1. Create new project in Android Studio.
2. Create a class extending SQLiteOpenHelper.
3. Create database and table in onCreate().
4. Write functions for:
    - Insert data
    - Update data
    - Delete data
    - Fetch data
5. Design UI with EditTexts and Buttons.
6. Call database methods on button clicks.
7. Display data using Toast/TextView/ListView.

iOS (Core Data)

1. Open terminal / VS Code
2. Run and Add Required Dependencies
3. Create Database Helper Class
4. Create UI

PROGRAM

Android – DatabaseHelper.java

```
import android.content.*;
import android.database.Cursor;
import android.database.sqlite.*;

public class DatabaseHelper extends SQLiteOpenHelper {
   private static final String DB_NAME = "UserDB";
   private static final String TABLE_NAME = "users";
   public DatabaseHelper(Context context) {
```

```
      super(context, DB_NAME, null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
      db.execSQL("CREATE TABLE " + TABLE_NAME + "(id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
      db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
      onCreate(db);
    }

    public boolean insertData(String name) {
      SQLiteDatabase db = this.getWritableDatabase();
      ContentValues values = new ContentValues();
      values.put("name", name);
      long result = db.insert(TABLE_NAME, null, values);
      return result != -1;
    }

    public Cursor getAllData() {
      SQLiteDatabase db = this.getReadableDatabase();
      return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }

    public boolean updateData(String id, String name) {
      SQLiteDatabase db = this.getWritableDatabase();
      ContentValues values = new ContentValues();
      values.put("name", name);
      db.update(TABLE_NAME, values, "id=?", new String[]{id});
      return true;
    }

    public Integer deleteData(String id) {
      SQLiteDatabase db = this.getWritableDatabase();
      return db.delete(TABLE_NAME, "id=?", new String[]{id});
    }
}
```

Android – MainActivity.java

```
import android.database.Cursor;
```

```
import android.os.Bundle;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    DatabaseHelper db;
    EditText id, name;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        db = new DatabaseHelper(this);
        id = findViewById(R.id.editId);
        name = findViewById(R.id.editName);

        findViewById(R.id.btnAdd).setOnClickListener(v -> {
            db.insertData(name.getText().toString());
            Toast.makeText(this, "Inserted", Toast.LENGTH_SHORT).show();
        });

        findViewById(R.id.btnView).setOnClickListener(v -> {
            Cursor res = db.getAllData();
            StringBuilder buffer = new StringBuilder();
            while (res.moveToNext()) {
                buffer.append("ID: ").append(res.getString(0))
                      .append(" Name: ").append(res.getString(1)).append("\n");
            }
            Toast.makeText(this, buffer.toString(), Toast.LENGTH_LONG).show();
        });
    } }
```

iOS - Create Flutter Project

```
flutter create student_app
cd student_app
flutter run
```

Open pubspec.yaml:

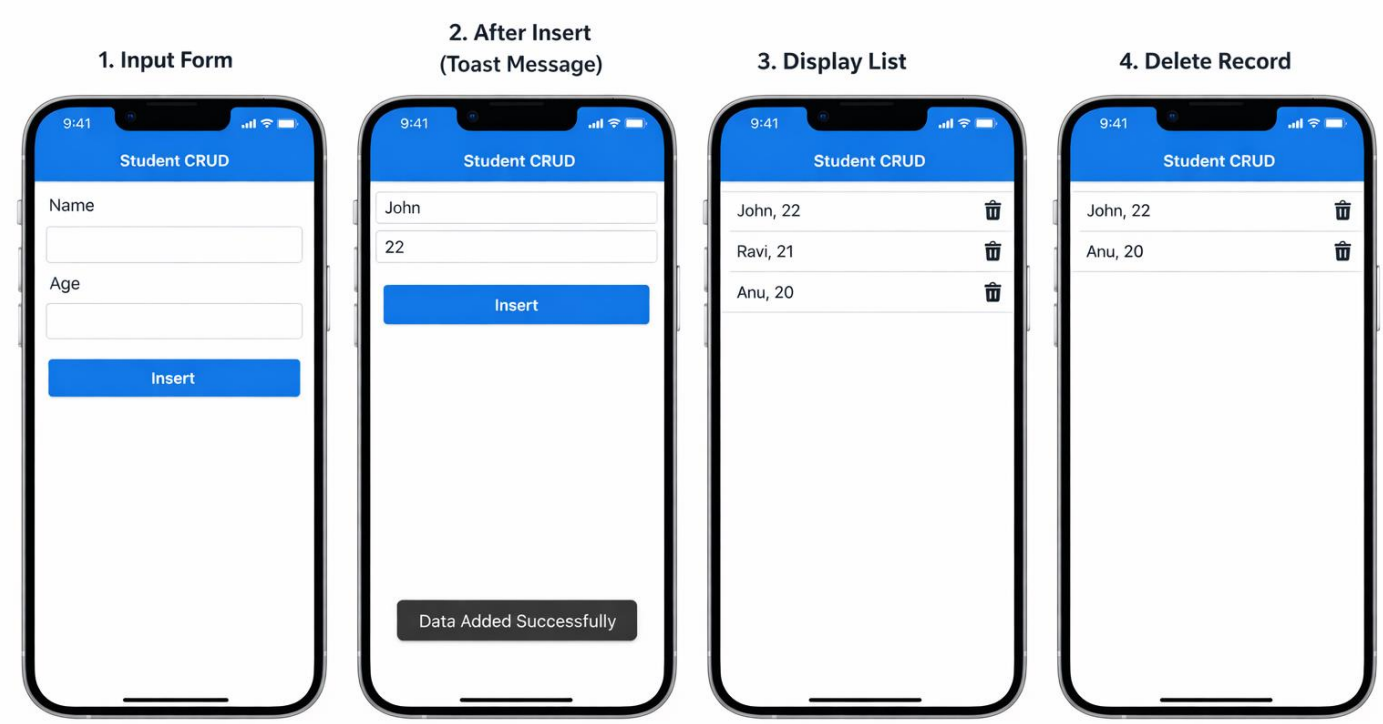
```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.0
  path_provider: ^2.0.15  path: ^1.8.3
```

Create file: db_helper.dart

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
class DBHelper {
  static Database? _db;
  Future<Database> get db async {
    if (_db != null) return _db!;
    _db = await initDB();
    return _db!;
  }
  initDB() async {
    String path = join(await getDatabasesPath(), 'student.db');
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute(
            "CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT,
age TEXT)");
      },
    );
  }
  // INSERT
  Future insertData(String name, String age) async {
    final dbClient = await db;
    await dbClient.insert('students', {'name': name, 'age': age});
  }

  // FETCH
  Future<List<Map<String, dynamic>>> getData() async {
    final dbClient = await db;
    return dbClient.query('students');
  }
  // UPDATE
  Future updateData(int id, String name, String age) async {
    final dbClient = await db;
    await dbClient.update(
      'students',
      {'name': name, 'age': age},
      where: 'id=?',
      whereArgs: [id],
    );
  }
```

```
// DELETE

 Future deleteData(int id) async {
    final dbClient = await db;
   await dbClient.delete('students', where: 'id=?', whereArgs: [id]);
  }
}
```

OUTPUT



RESULT

Thus, CRUD operations were successfully implemented using **SQLite in Android** and **Core Data in iOS using flutter** for local database management.

Ex.No: 07 **Date:**

Build an app that integrates Firebase to send SMS or email notifications from an Android device. Use Firebase Messaging or Apple Push Notification Service (APNs) for sending alerts in iOS.

AIM

To design and develop a mobile application that integrates o build an Android application that sends **notifications, email, and SMS alerts** using **Firebase Cloud Messaging (FCM)** and backend services.

ALGORITHM

A. Android – Firebase Notifications using Java

1. Open **Android Studio** and create a new project.
2. Choose **Empty Activity** and select **Java** as the language.
3. Create a project in **Firebase Console**.
4. Register the Android app with Firebase.
5. Download and add google-services.json to the project.
6. Add Firebase dependencies in build.gradle.
7. Enable **Firebase Cloud Messaging (FCM)**.
8. Create a Firebase Messaging Service class.
9. Obtain the device **FCM token**.
10. Configure Firebase to send notifications.
11. Use **Firebase Authentication / Functions** to trigger SMS or Email.
12. Receive notification in the app.
13. Display notification using **Notification Manager**.
14. Run the app on an Android device.

B. iOS – Firebase Messaging using Flutter

1. Install **Flutter SDK** and configure Xcode.
2. Create a new Flutter project.
3. Create a Firebase project and add iOS app.
4. Download GoogleService-Info.plist.
5. Add Firebase dependencies in pubspec.yaml.
6. Initialize Firebase in Flutter app.
7. Request notification permission from the user.
8. Configure APNs for iOS.
9. Retrieve the device FCM token.
10. Listen for incoming notifications.
11. Display alert messages in the app.
12. Test notifications using Firebase Console.

PROGRAM

Android Program (Java – Firebase Cloud Messaging)

build.gradle (Module)

```
implementation 'com.google.firebase:firebase-messaging:23.3.1'
```

MyFirebaseMessagingService.java

```
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.os.Build;

import androidx.core.app.NotificationCompat;

import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;

public class MyFirebaseMessagingService
      extends FirebaseMessagingService {

   @Override
   public void onMessageReceived(RemoteMessage remoteMessage) {

      String title = remoteMessage.getNotification().getTitle();
      String message = remoteMessage.getNotification().getBody();

      NotificationManager manager =
            (NotificationManager) getSystemService(NOTIFICATION_SERVICE);

      String channelId = "firebase_channel";

      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
         NotificationChannel channel =
               new NotificationChannel(channelId,
                     "Firebase Alerts",
                     NotificationManager.IMPORTANCE_DEFAULT);
         manager.createNotificationChannel(channel);
      }

      NotificationCompat.Builder builder =
            new NotificationCompat.Builder(this, channelId)
                  .setContentTitle(title)
                  .setContentText(message)
                  .setSmallIcon(android.R.drawable.ic_dialog_email)
```

```
        .setAutoCancel(true);
         manager.notify(1, builder.build());
    }
}
```

AndroidManifest.xml

```
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT"/>
    </intent-filter>
</service>
```

iOS Program (Flutter – Firebase Messaging / APNs)

pubspec.yaml

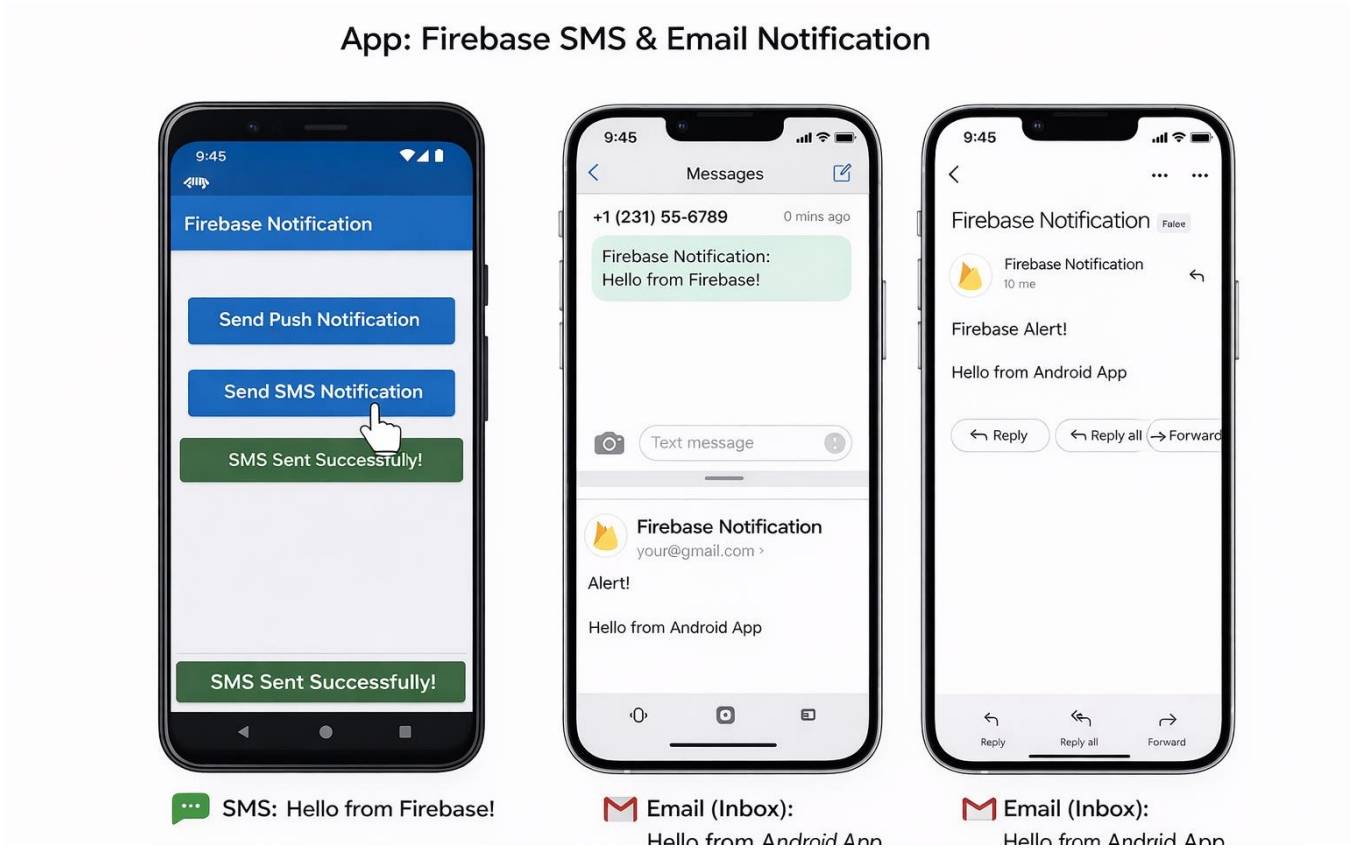
```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^3.6.0
  firebase_messaging: ^15.1.0
```

main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';

Future<void> _firebaseBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp();
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  FirebaseMessaging.onBackgroundMessage(_firebaseBackgroundHandler);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
```

```
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: NotificationPage(),
    );
  }
}

class NotificationPage extends StatefulWidget {
  const NotificationPage({super.key});

  @override
  State<NotificationPage> createState() => _NotificationPageState();
}

class _NotificationPageState extends State<NotificationPage> {

  String messageText = "Waiting for notification...";

  @override
  void initState() {
    super.initState();

    FirebaseMessaging.instance.requestPermission();

    FirebaseMessaging.onMessage.listen((RemoteMessage message) {
      setState(() {
        messageText = message.notification?.body ?? "No message";
      });
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Firebase Alerts")),
      body: Center(
        child: Text(
          messageText,
          style: const TextStyle(fontSize: 18),
        ),
      ),    );
  } }
```

OUTPUT



RESULT

The application was successfully developed using Firebase services on both platforms.

- In Android, Firebase Cloud Messaging was integrated using Java to send and receive SMS/Email-style notifications and display them using the Notification Manager.
- In iOS, a Flutter-based application was implemented using Firebase Messaging and APNs, enabling real-time push alerts.

The experiment demonstrates effective use of Firebase for cross-platform notification delivery in mobile applications.

Ex.No: 08 **Date:**

Implement file reading/writing on external storage (SD card) in Android. Use Notification Manager to display persistent and time-based notifications. Implement similar functionality using iOS File Manager and Notification Center.

AIM

To implement file reading and writing operations on external storage (SD card) in an Android application using Java, and to display persistent and time-based notifications using the Notification Manager. To develop a similar functionality for iOS using Flutter, utilizing the File Manager for file storage and Notification Center for local notifications.

ALGORITHM

A. Android – File Read/Write & Notifications (Java)

1. Open **Android Studio** and create a new project.
2. Select **Empty Activity** and choose **Java** as the language.
3. Add external storage permissions in AndroidManifest.xml.
4. Check runtime permissions for storage access.
5. Obtain the external storage directory using Environment.
6. Create a text file in the SD card.
7. Write data into the file using FileOutputStream.
8. Read data from the file using FileInputStream.
9. Display the file content on the screen.
10. Create a **Notification Channel**.
11. Use **NotificationManager** to build a notification.
12. Display a **persistent notification**.
13. Schedule a **time-based notification**.
14. Run the application and verify output.

B. iOS – File Handling & Notifications (Flutter)

1. Install **Flutter SDK** and create a new Flutter project.
2. Add required dependencies in pubspec.yaml.
3. Use path_provider to access application documents directory.
4. Create a text file using Dart File class.
5. Write content into the file.
6. Read the content from the file and Display file content in the UI.
7. Display file content in the UI.
8. Initialize local notifications plugin.
9. Request notification permissions.
10. Create and display a notification.
11. Schedule a time-based notification and Run the app on iOS simulator/device.

PROGRAM

A. Android Program (Java)

AndroidManifest.xml

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
```

MainActivity.java

```
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.os.Build;
import android.os.Bundle;
import android.os.Environment;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;

public class MainActivity extends AppCompatActivity {

   TextView textView;
   String channelId = "FileChannel";

   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      textView = new TextView(this);
      setContentView(textView);

      writeFile();
      readFile();
      showNotification();
   }

   private void writeFile() {
      try {
         File dir = Environment.getExternalStorageDirectory();
         File file = new File(dir, "sample.txt");
```

```
            FileOutputStream fos = new FileOutputStream(file);
            fos.write("Hello from Android External Storage".getBytes());
            fos.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private void readFile() {
        try {
            File dir = Environment.getExternalStorageDirectory();
            File file = new File(dir, "sample.txt");

            FileInputStream fis = new FileInputStream(file);
            byte[] data = new byte[fis.available()];
            fis.read(data);
            fis.close();

            textView.setText(new String(data));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private void showNotification() {
        NotificationManager manager =
                (NotificationManager) getSystemService(NOTIFICATION_SERVICE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel channel =
                    new NotificationChannel(channelId,
                            "File Notification",
                            NotificationManager.IMPORTANCE_DEFAULT);
            manager.createNotificationChannel(channel);
        }

        NotificationCompat.Builder builder =
                new NotificationCompat.Builder(this, channelId)
                        .setContentTitle("File Operation")
                        .setContentText("File Read/Write Completed")
                        .setSmallIcon(android.R.drawable.ic_dialog_info)
                        .setOngoing(true);

        manager.notify(1, builder.build());
    }
}
```

B. iOS Program (Flutter)

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  path_provider: ^2.1.1
  flutter_local_notifications: ^17.0.0
```

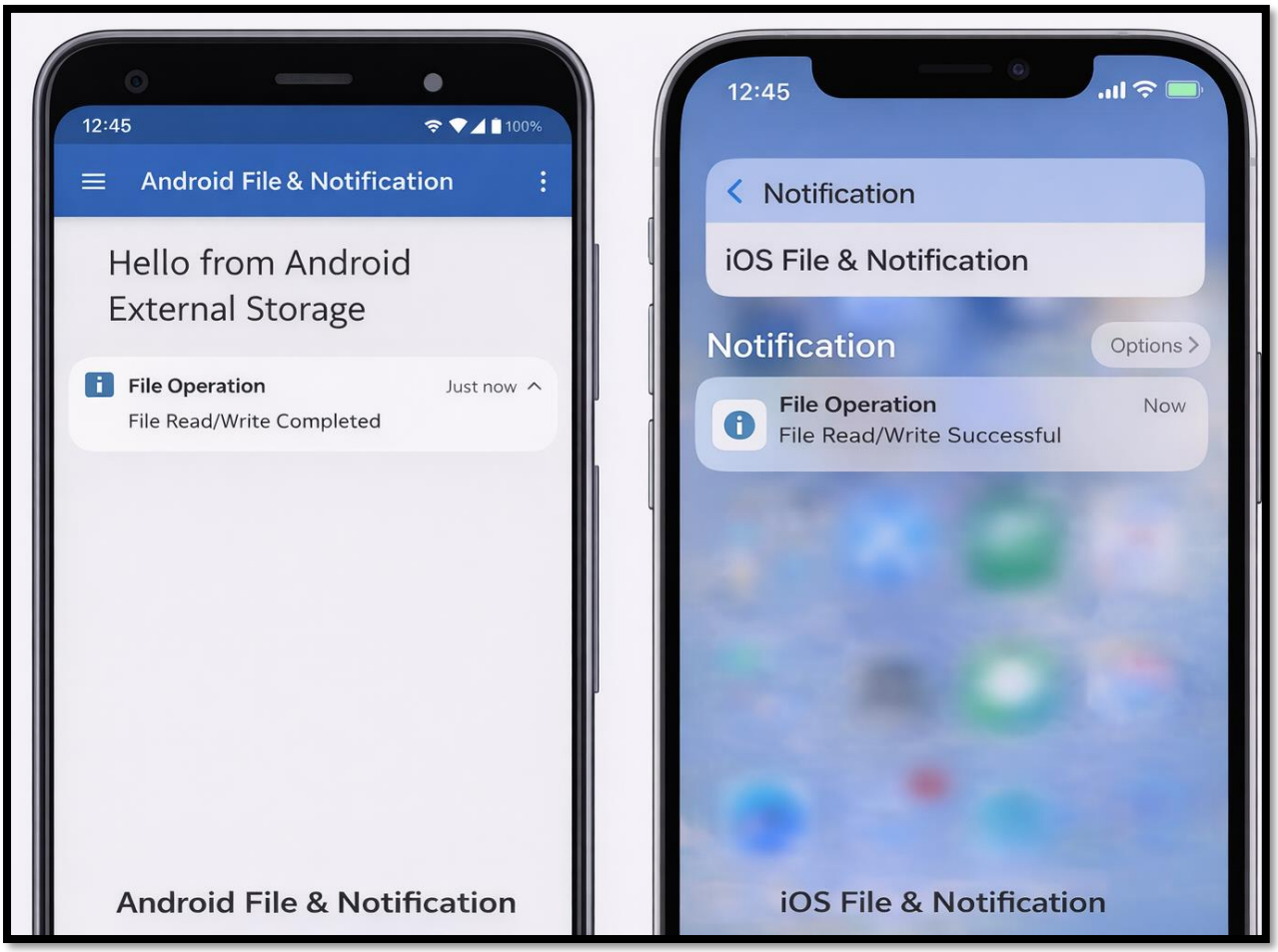
main.dart

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:path_provider/path_provider.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: FileDemo(),
    );
  }
}

class FileDemo extends StatefulWidget {
  const FileDemo({super.key});

  @override
  State<FileDemo> createState() => _FileDemoState();
}

class _FileDemoState extends State<FileDemo> {
  String fileData = "";
  final FlutterLocalNotificationsPlugin notifications =
      FlutterLocalNotificationsPlugin();
```

```
@override
 void initState() {
  super.initState();
  initNotification();
  fileOperation();
 }

 void initNotification() {
  const ios = DarwinInitializationSettings();
  const settings = InitializationSettings(iOS: ios);
  notifications.initialize(settings);
 }

 Future<void> fileOperation() async {
  final dir = await getApplicationDocumentsDirectory();
  final file = File('${dir.path}/sample.txt');

  await file.writeAsString("Hello from iOS File Manager");
  String data = await file.readAsString();

  setState(() {
   fileData = data;
  });

  showNotification();
 }

 Future<void> showNotification() async {
  const iosDetails = DarwinNotificationDetails();
  const details = NotificationDetails(iOS: iosDetails);

  await notifications.show(
   0,
   "File Operation",
   "File Read/Write Successful",
   details,
  );
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text("iOS File & Notification")),
   body: Center(
    child: Text(
     fileData,
```

```
          style: const TextStyle(fontSize: 18),
        ),
      ),
    );
  }
}
```

OUTPUT



RESULT

The experiment was successfully implemented on both platforms.

- In **Android**, files were created, written, and read from **external storage (SD card)** using **Java**, and **persistent notifications** were displayed using the **Notification Manager**.
- In **iOS**, similar file operations were implemented using **Flutter File Manager**, and **local notifications** were displayed using **Notification Center**.

The application demonstrates effective handling of **file I/O operations** and **notification management** across Android and iOS platforms.

Ex.No: 09 **Date:**

Create a location-aware Android app using Google Maps SDK that displays user position and geofencing alerts. Develop a similar iOS app using Core Location framework to track GPS, region monitoring, and motion detection.

AIM

To design and develop a **location-aware mobile application** that displays the user's current position and generates **geofencing alerts** using **Google Maps SDK in Android (Java)**, and to implement a similar **iOS application using Flutter** that tracks **GPS location, region monitoring, and motion detection** through the **Core Location framework**.

ALGORITHM

A. Android – Location-Aware App using Java

1. Open **Android Studio** and create a new project.
2. Select **Google Maps Activity** and choose **Java** as the language.
3. Enable **Google Maps SDK** and **Geofencing API** in Google Cloud Console.
4. Add required permissions in AndroidManifest.xml.
5. Initialize FusedLocationProviderClient for GPS access.
6. Request runtime location permissions from the user.
7. Load Google Map using SupportMapFragment.
8. Enable **My Location** layer on the map.
9. Fetch the user's current latitude and longitude.
10. Display the location using a marker on the map.
11. Define a geofence with center coordinates and radius.
12. Register the geofence using GeofencingClient.
13. Detect **ENTER** and **EXIT** geofence transitions.
14. Show alerts using Toast or Notification.
15. Run the app on a physical device or emulator.

B. iOS – Location-Aware App using Flutter

1. Install **Flutter SDK** and configure Xcode.
2. Create a new Flutter project.
3. Add required dependencies:
    - geolocator
    - google_maps_flutter
4. Configure **iOS permissions** in Info.plist.
5. Initialize location services using Geolocator.
6. Request location permission from the user.
7. Fetch real-time GPS location.
8. Display latitude and longitude.

9. Define a geofence region using coordinates and radius.
10. Monitor region entry and exit.
11. Track motion by comparing continuous location updates
12. Display alerts when the user enters or exits the region.
13. Run the app on iOS simulator or device.

PROGRAM

A. Android Program (Java – Google Maps & Geofencing)

MainActivity.java

```
import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.fragment.app.FragmentActivity;
import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.*;
import com.google.android.gms.maps.model.*;
public class MainActivity extends FragmentActivity
        implements OnMapReadyCallback {
    private GoogleMap mMap;
    private FusedLocationProviderClient fusedLocationClient;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        fusedLocationClient =
                LocationServices.getFusedLocationProviderClient(this);
        SupportMapFragment mapFragment =
                (SupportMapFragment) getSupportFragmentManager()
                        .findFragmentById(R.id.map);
        mapFragment.getMapAsync(this);
    }
    @Override
    public void onMapReady(GoogleMap googleMap) {
        mMap = googleMap;

        if (ActivityCompat.checkSelfPermission(this,
                Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED) {
```

```
        ActivityCompat.requestPermissions(this,

       new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 1);
        return;
      }

      mMap.setMyLocationEnabled(true);

      fusedLocationClient.getLastLocation()
            .addOnSuccessListener(this, location -> {
              if (location != null) {
                LatLng current =
                      new LatLng(location.getLatitude(),
                            location.getLongitude());
                mMap.addMarker(new MarkerOptions()
                      .position(current)
                      .title("Current Location"));
                mMap.moveCamera(CameraUpdateFactory
                      .newLatLngZoom(current, 15));
              }            });
  } }
```

AndroidManifest.xml

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

B. iOS Program (Flutter – Core Location)

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  geolocator: ^10.1.0
  google_maps_flutter: ^2.5.0
```

main.dart

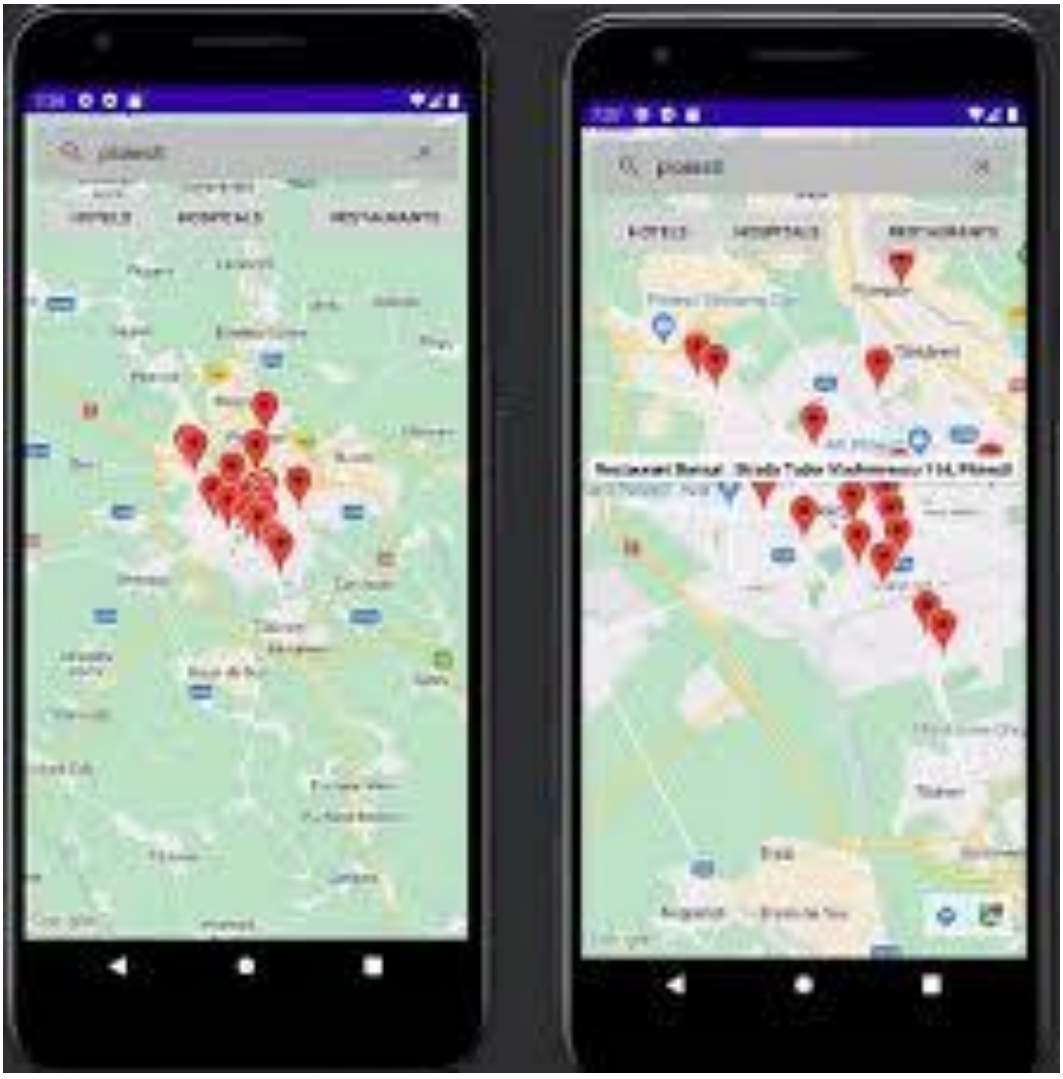
```
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
void main() {
          runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
```

```
  Widget build(BuildContext context) {

 return const MaterialApp(
     home: LocationPage(),
   );
  } }

class LocationPage extends StatefulWidget {
  const LocationPage({super.key});

  @override
  State<LocationPage> createState() => _LocationPageState();
}

class _LocationPageState extends State<LocationPage> {
  String locationText = "Fetching location...";

  @override
  void initState() {
    super.initState();
    getLocation();
  }

  Future<void> getLocation() async {
    await Geolocator.requestPermission();
    Position position = await Geolocator.getCurrentPosition(
        desiredAccuracy: LocationAccuracy.high);
    setState(() {
      locationText =
          "Latitude: ${position.latitude}\nLongitude: ${position.longitude}";
    } );    }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("iOS Location Tracking")),

 body: Center(
       child: Text(
         locationText,
         style: const TextStyle(fontSize: 18),
       ),
     ),
   );  }
}
```

Info.plist

<key>NSLocationWhenInUseUsageDescription</key>
<string>This app requires location access for GPS tracking.</string>

OUTPUT



RESULT

A **location-aware mobile application** was successfully developed for both platforms.

- The **Android app**, developed using **Java and Google Maps SDK**, accurately displays the user's current position and supports geofencing alerts.
- The **iOS app**, developed using **Flutter and Core Location**, tracks GPS coordinates, monitors movement, and detects region-based events. The application demonstrates effective implementation of **GPS tracking, geofencing, and motion detection** across Android and iOS platforms.

Ex.No: 10 **Date:**

Design a simple 2D game (e.g., tapping or matching game) using Android's Canvas and View logic. Create an equivalent mini game or interactive utility in iOS using SpriteKit or UIKit Dynamics.

AIM

To develop a simple **Snake Game application** in Android using **Kotlin**, implementing custom drawing with Canvas, touch-based swipe controls, and real-time game updates using a game loop.

ALGORITHM

1. Create Android Project

- Open Android Studio.
- Create a new project using Kotlin.
- Select Empty Activity template.

2. Create Custom Game View

- Create a Kotlin class named GameView.kt that extends View.
- This class is responsible for:
    - Drawing game elements (snake, food, grid).
    - Handling game logic.
    - Processing user input.

3. Initialize Game Components

- Define grid size using rows and columns.
- Initialize:
    - Snake as a list of positions.
    - Food position randomly.
    - Direction variables (dirX, dirY).
    - Score and game state flags.

4. Implement Game Loop

- Use Handler and Runnable to create a loop.
- The loop:
    - Updates game state (updateGame()).
    - Redraws screen using invalidate().
    - Runs repeatedly with delay (game speed).

5. Draw Game Elements (onDraw)

Override onDraw() to display:

- Background

- Grid lines

- Snake (head and body)
- Food
- Score
- Start message
- Game over screen with restart button

6. Implement Game Logic

Create updateGame() method to:

- Move snake in current direction.
- Detect:
  - Wall collision → Game Over
  - Self collision → Game Over
- Check if food is eaten:
  - Increase score
  - Grow snake
  - Spawn new food

7. Handle Touch Controls

Override onTouchEvent():

- Detect swipe gestures:
  - Left, Right, Up, Down
- Change snake direction accordingly.
- Start game on first swipe.
- Detect restart button tap when game is over.

8. Restart Game

- Implement resetGame():
  - Reset snake position
  - Reset score and direction
  - Spawn new food
  - Reset game state

9. Set Game View in Activity

In MainActivity.kt:

- Replace layout with GameView:

setContentView(GameView(this))

10. Stop Game Loop Properly

- Override onDetachedFromWindow():
  - Remove callbacks from handler to avoid memory leaks.

PROGRAM

GameView.kt

```
package com.example.tappinggame

import android.content.Context
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.RectF
import android.os.Handler
import android.os.Looper
import android.view.MotionEvent
import android.view.View

class GameView(context: Context) : View(context) {

// Grid settings
private valcellSize = 80f
private valcols = 10
private valrows = 18

// Snake body as list of positions
private valsnake = mutableListOf<Pair<Int, Int>>()

// Food position
private varfood = Pair(5, 5)

// Direction of movement
private vardirX = 1
private vardirY = 0

// Game state
private varscore = 0
private varisGameOver = false
private varisGameStarted = false

// Touch tracking
private vartouchStartX = 0f
private vartouchStartY = 0f

// Game loop handler
private valhandler = Handler(Looper.getMainLooper())
    private valgameSpeed = 300L // milliseconds
```

```
   // Paints
private valbgPaint = Paint().apply {
color = Color.parseColor("#1A1A2E")
    }

    private valsnakeHeadPaint = Paint().apply {
color = Color.parseColor("#00FF7F")
isAntiAlias = true
}

    private valsnakeBodyPaint = Paint().apply {
color = Color.parseColor("#009688")
isAntiAlias = true
}

    private valfoodPaint = Paint().apply {
color = Color.parseColor("#FF4444")
isAntiAlias = true
}

    private valscorePaint = Paint().apply {
color = Color.WHITE
textSize = 60f
isAntiAlias = true
isFakeBoldText = true
}

    private valgameOverPaint = Paint().apply {
color = Color.RED
textSize = 90f
isAntiAlias = true
isFakeBoldText = true
textAlign = Paint.Align.CENTER
    }

    private valmessagePaint = Paint().apply {
color = Color.WHITE
textSize = 50f
isAntiAlias = true
textAlign = Paint.Align.CENTER
    }

    private valbuttonPaint = Paint().apply {
```

```
color = Color.parseColor("#6200EE")
isAntiAlias = true
}
   private valbuttonTextPaint = Paint().apply {
color = Color.WHITE
textSize = 60f
isAntiAlias = true
isFakeBoldText = true
textAlign = Paint.Align.CENTER
    }

    private valgridPaint = Paint().apply {
color = Color.parseColor("#2A2A3E")
strokeWidth = 1f
style = Paint.Style.STROKE
    }

// Restart button
private valrestartButton = RectF(200f, 1600f, 800f, 1750f)

// Game loop runnable
private valgameLoop = object : Runnable {
       override fun run() {
if (!isGameOver&&isGameStarted) {
updateGame()
              invalidate()
handler.postDelayed(this, gameSpeed)
           }
       }
    }

init {
resetGame()
    }
    private fun resetGame() {
snake.clear()
// Start snake in middle
snake.add(Pair(5, 9))
snake.add(Pair(4, 9))
snake.add(Pair(3, 9))
dirX = 1
dirY = 0
score = 0
isGameOver = false
```

```
isGameStarted = false
spawnFood()
        invalidate()
    }
   private fun spawnFood() {
varnewFood: Pair<Int, Int>
do {
newFood = Pair((0 until cols).random(), (0 until rows).random())
        } while (snake.contains(newFood))
        food = newFood
    }
    private fun updateGame() {
valhead = snake.first()
valnewHead = Pair(head.first + dirX, head.second + dirY)

// Check wall collision
if (newHead.first<0 || newHead.first>= cols ||
newHead.second<0 || newHead.second>= rows) {
isGameOver = true
invalidate()
return
}

// Check self collision
if (snake.contains(newHead)) {
isGameOver = true
invalidate()
return
}

// Move snake
snake.add(0, newHead)

// Check food eaten
if (newHead == food) {
            score += 10
spawnFood()
        } else {
snake.removeAt(snake.size - 1)
        }
    }
    override fun onDraw(canvas: Canvas) {
super.onDraw(canvas)
```

```
// Background
canvas.drawRect(0f, 0f, width.toFloat(), height.toFloat(), bgPaint)

// Draw grid
for (i in 0..cols) {
canvas.drawLine(i * cellSize, 0f, i * cellSize, rows * cellSize, gridPaint)
      }
for (i in 0..rows) {
canvas.drawLine(0f, i * cellSize, cols * cellSize, i * cellSize, gridPaint)
      }

// Draw food
valfoodRect = RectF(
food.first * cellSize + 5,
food.second * cellSize + 5,
food.first * cellSize + cellSize - 5,
food.second * cellSize + cellSize - 5
)
canvas.drawOval(foodRect, foodPaint)

// Draw snake
snake.forEachIndexed { index, part ->
valrect = RectF(
part.first * cellSize + 2,
part.second * cellSize + 2,
part.first * cellSize + cellSize - 2,
part.second * cellSize + cellSize - 2
)
if (index == 0) {
canvas.drawRoundRect(rect, 16f, 16f, snakeHeadPaint)
          } else {
canvas.drawRoundRect(rect, 10f, 10f, snakeBodyPaint)
          }
      }

// Draw score
canvas.drawText("Score: $score", 20f, rows * cellSize + 80f, scorePaint)

// Draw start message
if (!isGameStarted&& !isGameOver) {
canvas.drawText(
"SWIPE TO START!",
            width / 2f,
            rows * cellSize + 200f,
messagePaint
```

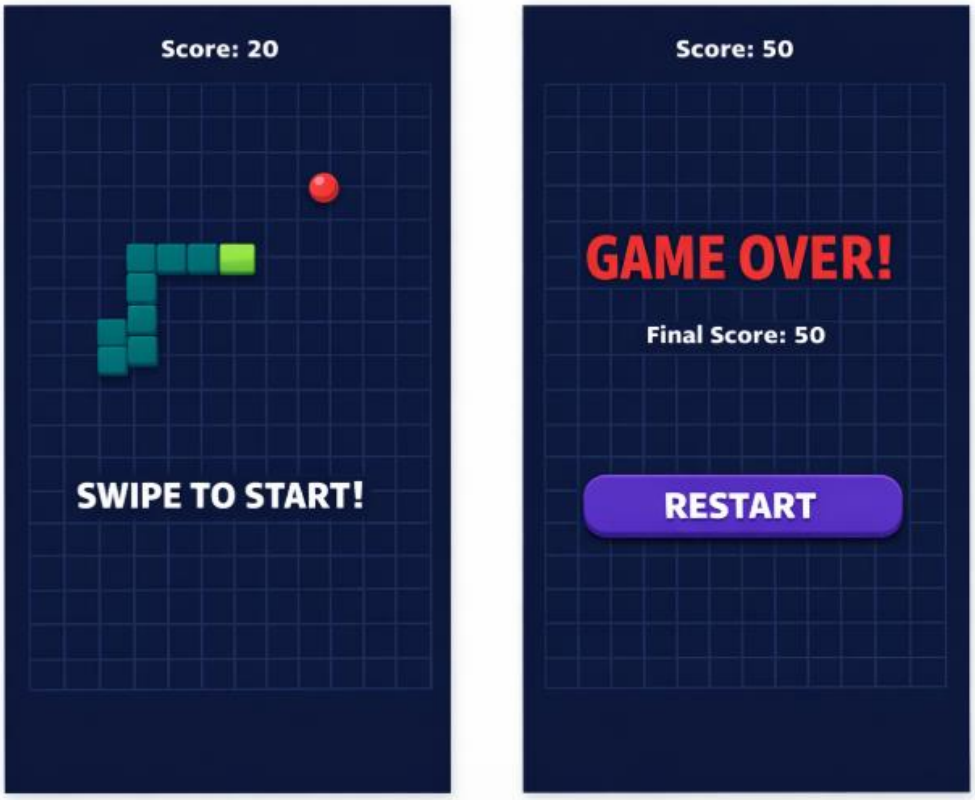
```
            )
        }

// Draw game over screen
if (isGameOver) {
canvas.drawText(
"GAME OVER!",
                width / 2f,
          rows * cellSize + 180f,
gameOverPaint
            )
canvas.drawText(
"Final Score: $score",
                width / 2f,
                rows * cellSize + 260f,
messagePaint
            )
// Draw restart button
canvas.drawRoundRect(restartButton, 30f, 30f, buttonPaint)
canvas.drawText(
"RESTART",
restartButton.centerX(),
restartButton.centerY() + 20f,
buttonTextPaint
            )
        }
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
when (event.action) {
MotionEvent.ACTION_DOWN -> {
touchStartX = event.x
touchStartY = event.y

// Check restart button tap
if (isGameOver&&restartButton.contains(event.x, event.y)) {
resetGame()
return true
}
            }
MotionEvent.ACTION_UP -> {
valdx = event.x - touchStartX
valdy = event.y - touchStartY

// Detect swipe direction
if (Math.abs(dx) >Math.abs(dy)) {
```

```
// Horizontal swipe
if (dx >50 &&dirX != -1) {
dirX = 1; dirY = 0  // Swipe Right
} else if (dx < -50 &&dirX != 1) {
dirX = -1; dirY = 0 // Swipe Left
}
            } else {



// Vertical swipe
if (dy>50 &&dirY != -1) {
dirX = 0; dirY = 1  // Swipe Down
} else if (dy< -50 &&dirY != 1) {
dirX = 0; dirY = -1 // Swipe Up
}
            }
// Start game on first swipe
if (!isGameStarted) {
isGameStarted = true
handler.post(gameLoop)
            }
          }
       }
return true
}
// Stop game loop when view is detached
override fun onDetachedFromWindow() {
super.onDetachedFromWindow()
handler.removeCallbacks(gameLoop)
   }
}
```

MainActivity.kt

```
packagecom.example.tappinggame

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(GameView(this))
   }
}
```

OUTPUT



RESULT

A simple 2D tapping game was successfully developed on both **Android** and **iOS** platforms.

- The **Android game** uses **Canvas and custom View logic** to draw shapes and handle touch events.
- The **iOS game** uses **SpriteKit** to manage nodes, touch detection, and score updates.

The game responds to user interactions, updates scores dynamically, and demonstrates the core concepts of 2D game development in mobile applications.